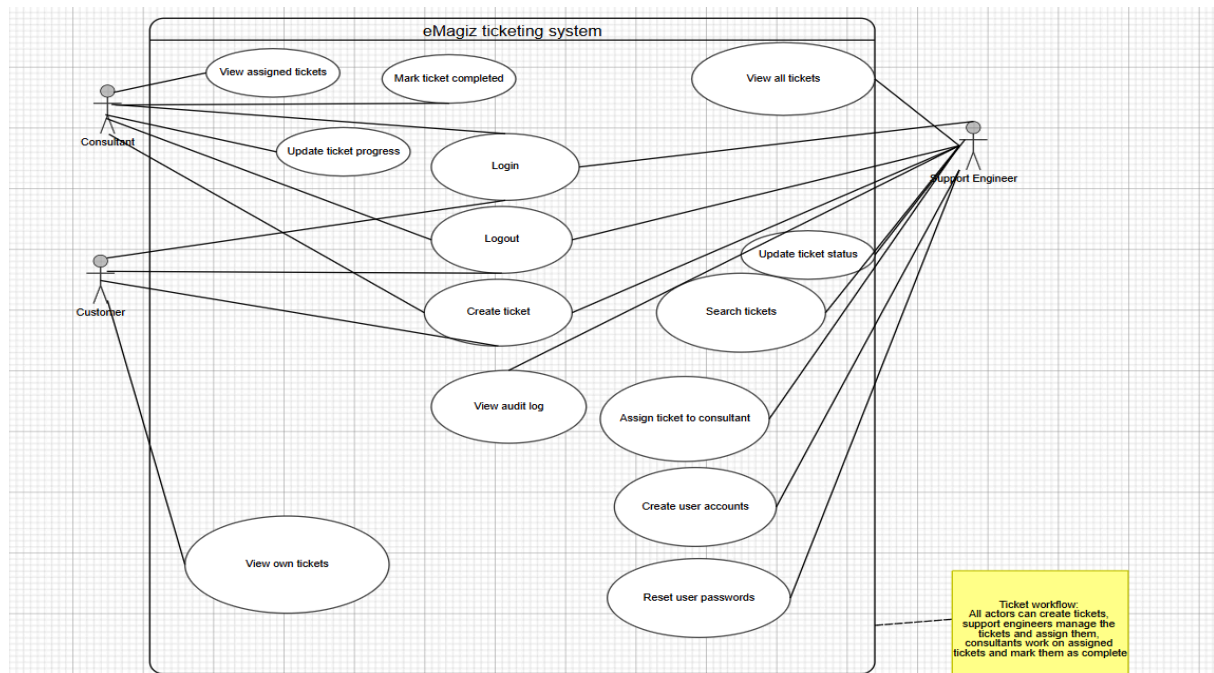


Use Case Diagram



Our ticket management system has three main actors: Customer, Consultant, and Support.

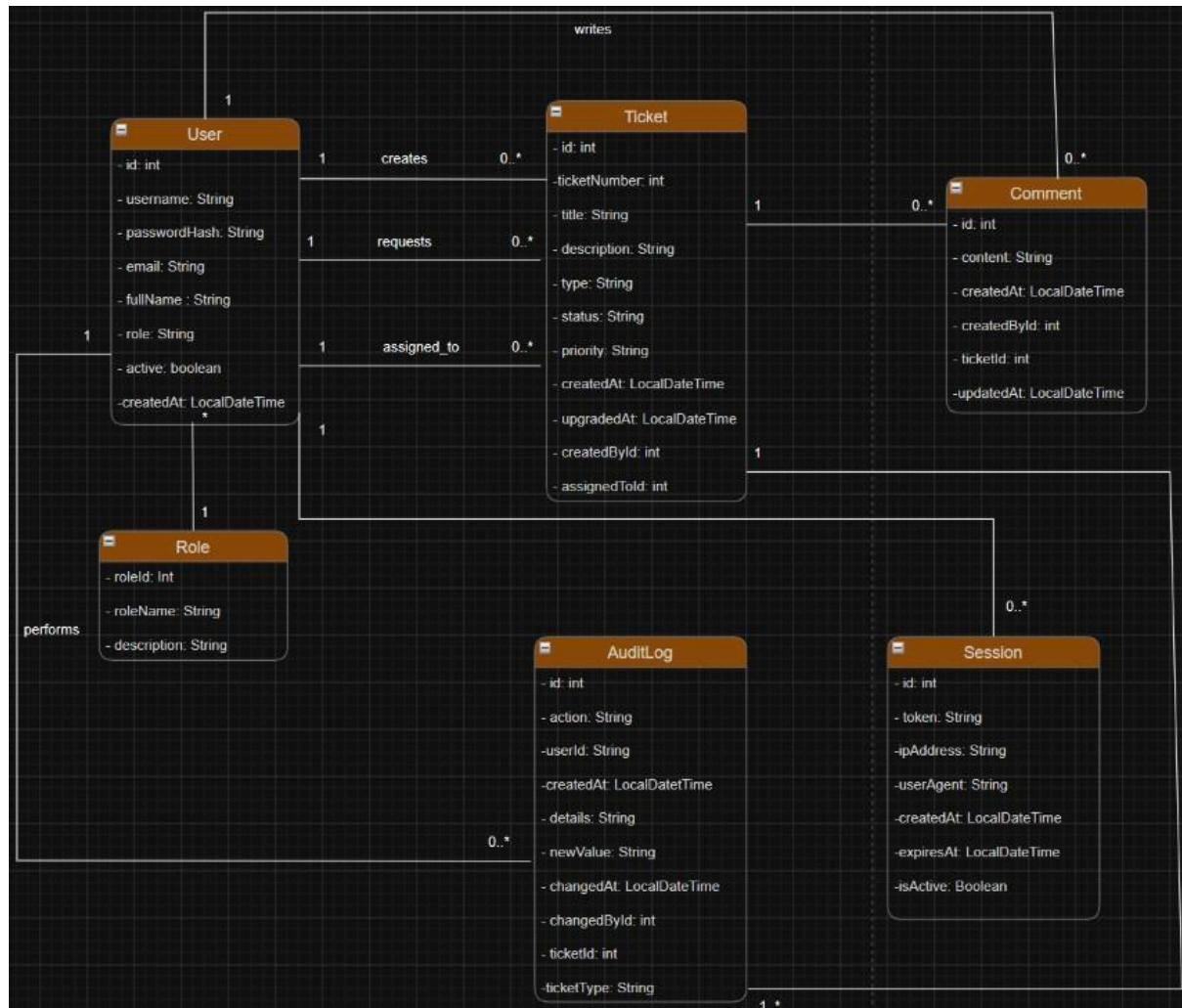
The Customer can log in, view their own profile, create Incident and RFC tickets, view only the tickets they have created, track the progress of those tickets, and add comments where required. Customers cannot view other users' tickets or perform administrative actions.

The Support role acts as the administrator of the system. Support users can log in, view every ticket in the system, create customer accounts, manage customer accounts, assign and reassign tickets to consultants, update ticket status, view audit logs, and search the audit history. They are responsible for managing the overall ticket workflow.

The Consultant can log in, view only the tickets assigned to them, accept or reject assigned tickets, update ticket progress, change ticket status during the workflow, and add comments while working on a ticket.

The updated use case diagram also includes Login, Logout, and Customer Account Management functionality, which were added based on the feedback received after the previous assignment.

Class Diagram



The class diagram represents the main classes used in our ticket management system and the relationships between them.

The central class is User, which stores information such as the user ID, username, password hash, email, full name, role, account status, and creation date. Instead of using subclasses, each user is linked to a Role class, which defines whether the user is a Customer, Consultant, or Support user.

A User can create many Tickets, and consultants can also be assigned many tickets. To represent these two different relationships, the Ticket class contains separate references for createdBy and assignedTo. Each ticket stores information such as its ticket number, title, description, type, priority, status, creation date, last updated date, creator, and assigned consultant.

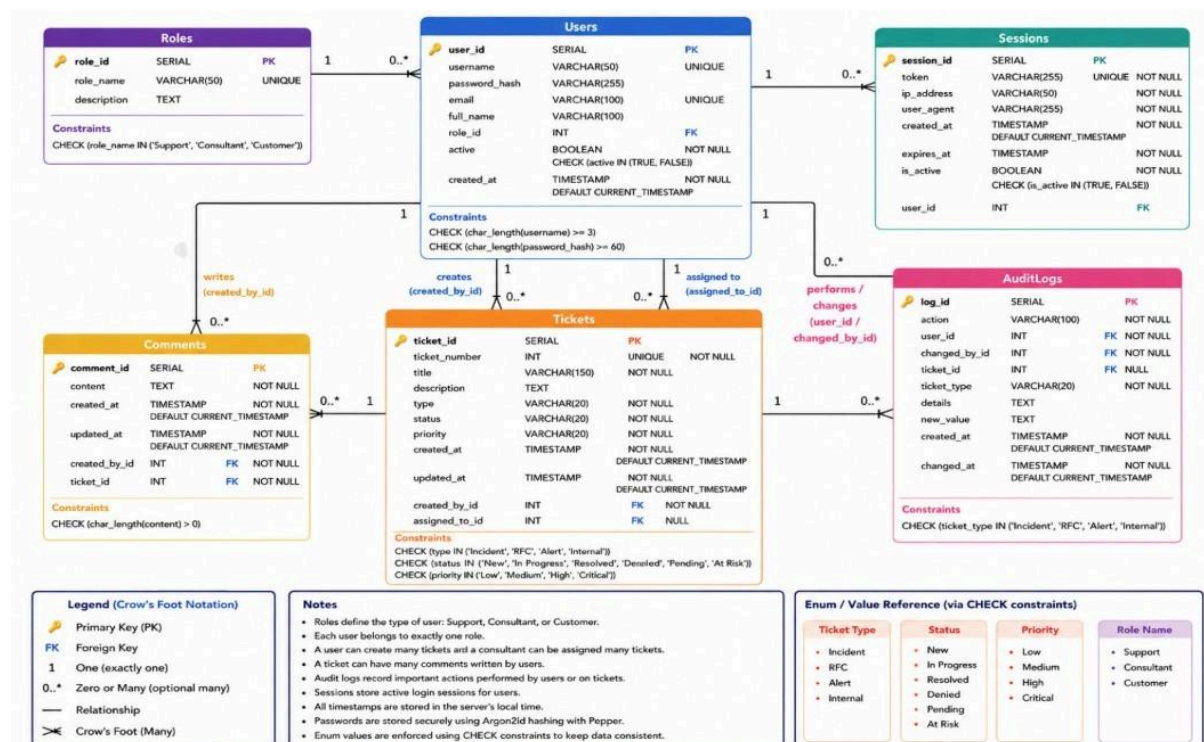
To support communication between users, we added a separate Comment class. Every comment belongs to one ticket and is written by one user, allowing multiple comments to be attached to a ticket throughout its lifecycle.

We also introduced an AuditLog class to record important actions performed in the system, including ticket updates, assignments, login events, and account changes. Each audit record stores details about the action, the user who performed it, the affected ticket, timestamps, and any relevant changes.

Finally, a Session class models active login sessions by storing session tokens, expiry times, IP addresses, user agents, and session status for authenticated users.

Overall, the updated class diagram now models the complete structure of the application much more accurately and reflects the relationships used by both the frontend and backend.

SQL Schema



The SQL schema stores and manages all of the application's data using a relational database.

The database consists of tables for Roles, Users, Tickets, Comments, AuditLogs, and Sessions. Relationships between these tables ensure that users, tickets, comments, sessions, and audit records remain connected while maintaining data integrity through primary keys and foreign keys.

The Roles table is used instead of storing role names directly inside the Users table, making role management cleaner and easier to maintain. Ticket fields such as Type, Status, and Priority are validated using database CHECK constraints, ensuring only valid values can be stored.

The Comments table was added so that multiple comments can be attached to each ticket while recording the user who created the comment and the associated timestamps.

The AuditLogs table records important system activities such as logins, ticket creation, ticket updates, assignments, and account management actions, providing a complete audit trail.

The Sessions table stores active login sessions, including session tokens, expiration times, IP addresses, and user-agent information for authenticated users.

The backend communicates with the database through JDBC and SQL queries for tasks such as user authentication, ticket creation, ticket assignment, audit logging, session management, comments, and displaying ticket information throughout the application.

During development we initially used PostgreSQL, and the project has now been migrated to the University of Twente database while keeping the same relational design..